**INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA**

**KULLIYYAH OF ENGINEERING**

**MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)**

**SEMESTER 2 2025/2026**

**WEEK 3: SERVOMOTOR**

**SECTION 1**

**GROUP 11**

**INSTRUCTOR: DR. WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL ABIDIN**

**Date of Experiment:** Wednesday, 18$^{TH}$ March 2026

**Date of Submission:** Wednesday, 25$^{TH}$ March 2026

| NO. | NAME | MATRIC NO. |
|---|---|---|
| 1. | NURUL ANIS BINTI MOHD KHAIRI | 2419788 |
| 2. | WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN | 2320102 |
| 3. | SITI NURKHALISAH BINTI NAHZAMRI | 2328482 |

## ABSTRACT

This experiment presents the integration of Arduino and Python for real-time control, bidirectional serial communication, and data visualisation in a servomotor control system. The main objective of this experiment was to transmit sensor data from Arduino to Python, implement bidirectional communication between both platforms, and perform real-time control and monitoring of a servo motor using a potentiometer input. The experiment was divided into two tasks, where Task 1 focused on bidirectional communication and LED control based on potentiometer readings, while Task 2 focused on real-time servo motor control and graphical data visualisation.

In Task 1, the potentiometer readings were successfully transmitted from Arduino to Python via USB serial communication, and Python sent commands back to Arduino to control the LED based on a predefined threshold. The LED turned ON when the voltage exceeded approximately 2.5 V, demonstrating that an analog input signal can be used to control a digital output through conditional logic. In Task 2, the Arduino continuously mapped the potentiometer input to servo motor angles and transmitted the angle data to Python for real-time plotting using matplotlib. The system also implemented bidirectional communication, where Python could send a stop command to safely terminate the operation. Overall, the experiment successfully demonstrated hardware-software integration, real-time control, serial communication, and automated decision-making in a simple mechatronic control system.

# Table of Contents

Table of Contents ..... 3
INTRODUCTION ..... 4
TASK 1 ..... 5
1.1 MATERIALS AND EQUIPMENTS ..... 5
1.2 EXPERIMENTAL SETUP ..... 5
1.3 METHODOLOGY ..... 5
**a. Circuit Assembly** ..... 5
**b. Programming and Uploading** ..... 6
**c. Programming Logic** ..... 6
**d. Coding** ..... 6
1.4 DATA COLLECTION ..... 8
1.5 DATA ANALYSIS ..... 9
1.6 RESULT ..... 9
1.7 DISCUSSION ..... 10
1.8 CONCLUSION ..... 10
TASK 2 ..... 11
2.1 MATERIALS AND EQUIPMENTS ..... 11
2.2 EXPERIMENTAL SETUP ..... 11
2.3 METHODOLOGY ..... 12
**a. Circuit Assembly** ..... 12
**b. Programming and Uploading** ..... 12
**c. Programming Logic** ..... 12
**d. Coding** ..... 13
2.4 DATA COLLECTION ..... 15
2.5 DATA ANALYSIS ..... 15
2.6 RESULT ..... 16
2.7 DISCUSSION ..... 17
2.8 CONCLUSION ..... 18
3.0 RECOMMENDATION ..... 19
4.0 REFERENCES ..... 20
5.0 APPENDICES ..... 20
6.0 ACKNOWLEDGEMENT ..... 21
7.0 STUDENTS DECLARATION ..... 21
***Certificate of Originality and Authenticity*** ..... 21

## INTRODUCTION

Servomotors are an electric motor that provides precise control of angular or linear position, speed and torque using a feedback loop system. It uses a closed-loop servomechanism that utilises position feedback to control its motion and final position, allowing for accurate control of angular position. Therefore, a servomotor usually consists of a motor, a feedback system, and a controller that monitors the motor's actual position and adjusts it to match the desired position. They are widely used in control systems, robotics, and automation applications due to their ability to provide precise control of angular position. Thus, this experiment focuses on the integration of hardware and software through serial communication, sensor input processing, actuator control, and real-time data visualisation. This experiment also demonstrates that the analog input devices, output components, and software visualisation tools can be combined to form a simple real-time control system.

For Task 1, the experiment focuses on generating a bidirectional serial communication between the Arduino and Python while monitoring analog input from a potentiometer. The potentiometer then acts as a variable input device where its analog voltage is converted into digital values using Arduino's Analog-to-Digital Converter (ADC). The Arduino sends the potentiometer value to Python, and Python processes the data and sends a command back to Arduino to control an LED. The LED turns ON when the potentiometer value exceeds half of its maximum ADC range and turns OFF when the value is below the threshold. This task demonstrates the concepts of ADC conversion, serial communication, sensor data acquisition, and conditional output control.

For Task 2, the experiment extends the system to real-time servomotor control and data visualisation. A servomotor is connected to the Arduino and controlled by mapping the potentiometer input to the servo angle, while an LED is used as an indicator when the servo angle exceeds 90 degrees. The Arduino continuously sends the servo angle data to Python through serial communication, and Python visualizes the data in real time using the matplotlib library. The system can be safely stopped using a keyboard interrupt command. This task demonstrates real-time control systems, servo motor operation, hardware-software integration, and live data visualization.

# TASK 1

## 1.1 MATERIALS AND EQUIPMENTS

- Arduino board
- Potentiometer
- LED
- 220 Ω resistor
- Jumper wires
- Breadboard

## 1.2 EXPERIMENTAL SETUP

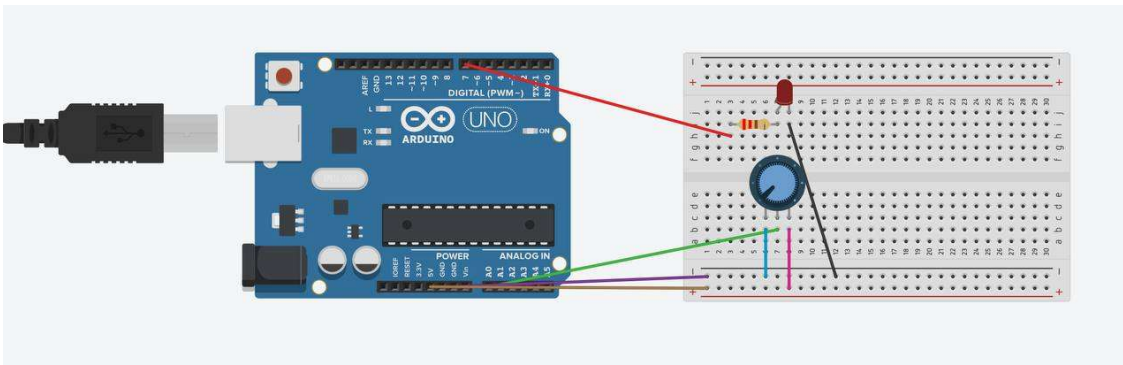



*Figure 1 illustrates the task 1 setup constructed using tinker cad*

1. The potentiometer was connected to the Arduino as follows:
   - One end connected to 5V
   - Another end connected to GND pin
   - Center pin connected to an analog input pin A0 on the Arduino
2. An LED was added to digital pin 7 in series with a 220 Ω resistor connected to GND.
3. Arduino code was written to read the potentiometer value and send it to Python, and a command will be sent back to turn on the LED ON if value exceeds half its maximum range.

## 1.3 METHODOLOGY

### a. Circuit Assembly

- All components were assembled based on the given circuit setup and wiring instructions (Fig.1).
- The wiring was ensured for proper continuity and to prevent short circuits.
- The Arduino Uno was connected to the computer by using a USB cable for power and data transfer.

### b. Programming and Uploading

- The Arduino IDE software was opened on the computer.
  - The correct board type (Arduino Uno) and communication port (COM port 9) were selected in the Tools menu.
  - The coding was uploaded and compiled with no syntax errors.

- The latest version of Python and PyCharm were downloaded on the computer.
  - During installation, ensure to check the box that says, “Add Python to PATH”.
  - Install the required libraries by typing this single command and pressing Enter:
    `pip install pyserial matplotlib`
  - The Python script was uploaded and executed. To stop the program safely, click the red “Stop” square in PyCharm or press CTRL+C in the terminal window.

- Ensure that Arduino IDE is not opened when running Python code in PyCharm.

### c. Programming Logic

- Arduino IDE
  - The potentiometer and LED pins were initialized.
  - The potentiometer was checked continuously by the program and send the value to the Python.
  - Python will send back a command to the Arduino.
  - When the value of potentiometer exceeds half of its maximum range (511.5), the LED will turn ON. Otherwise, the LED will turn OFF.

- Python (PyCharm)
  - The value of potentiometer was sent to the PyCharm.
  - The value of voltage, resistance and Analog-to-Digital Converter (ADC) were displayed on serial monitor.
  - A command was sent to Arduino to either tur ON or OFF the LED.

### d. Coding

- Arduino code

```
const int potPin = A0;
const int ledPin = 7;
const float Vin = 5.0;
const float R_total = 10000.0;
String command = "";

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
}

void loop() {
  int potValue = analogRead(potPin);
  float Vout = (potValue / 1023.0) * Vin;
  float R_wiper = (Vout / Vin) * R_total;

  // Send formatted data to Python
  Serial.print(Vout);
  Serial.print(" / ADC: ");
  Serial.print(potValue);
  Serial.print(" / R: ");
  Serial.println(R_wiper);

  // Check for incoming commands from Python
  if (Serial.available() > 0) {
    command = Serial.readStringUntil('\n');
    command.trim();

    if (command == "LED ON") {
      digitalWrite(ledPin, HIGH);
    }
    else if (command == "LED OFF") {
      digitalWrite(ledPin, LOW);
    }
  }
  delay(500);
}
```

- Python script (PyCharm)

```python
import serial
import time

ser = serial.Serial( port: 'COM9',  baudrate: 9600)
time.sleep(2)  # Wait for Arduino to reset

print("Running... Press CTRL+C to stop.")

try:
    while True:
        line = ser.readline().decode().strip()
        if "/" in line:
            try:
                # Parse Arduino data
                data = line.split("/")
                voltage = float(data[0])
                adc_value = int(data[1].replace( old: "ADC:",  new: "").strip())
                resistance = float(data[2].replace( old: "R:",  new: "").strip())

                print(f"Voltage: {voltage:.2f} V | ADC: {adc_value} | Resistance: {resistance:.2f} Ω")

                # LED Control Logic
                if adc_value > (1023 / 2):
                    ser.write(b"LED ON\n")
                else:
                    ser.write(b"LED OFF\n")
            except Exception as e:
                print("Parsing Error:", line, "->", e)

except KeyboardInterrupt:
    print("\nStopped by user.")
finally:
    ser.close()
    print("Serial connection closed.")
```

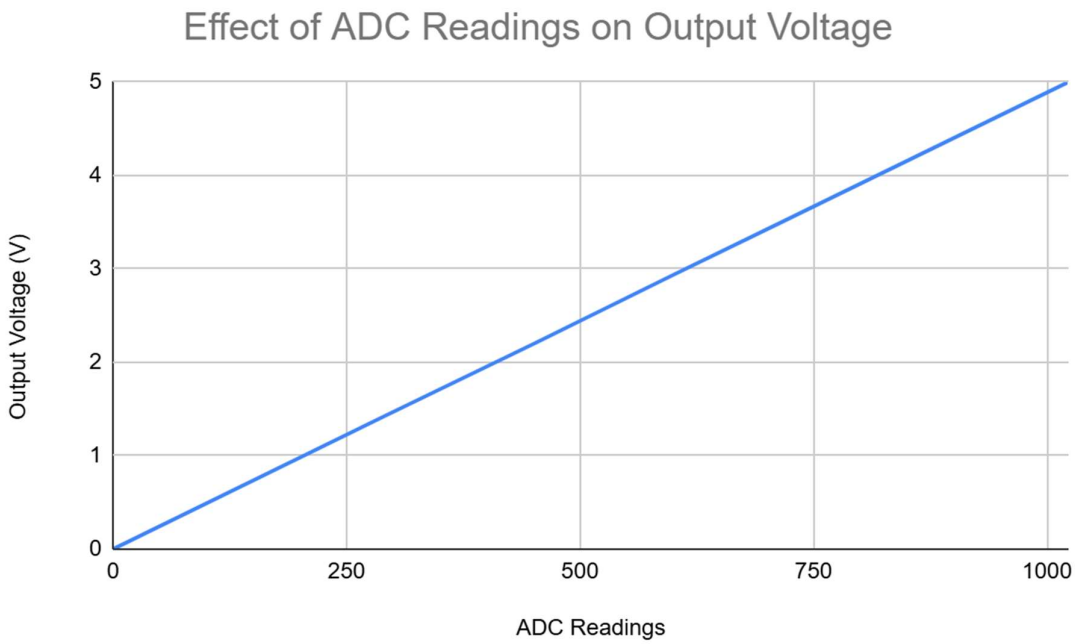
## 1.4 DATA COLLECTION

| Trial | Potentiometer Value (0-1023) | Output Voltage (V) | LED State |
|---|---|---|---|
| 1 | 0 | 0 | OFF |
| 2 | 292 | 1.43 | OFF |
| 3 | 451 | 2.20 | OFF |
| 4 | 601 | 2.94 | ON |
| 5 | 720 | 3.52 | ON |
| 6 | 1023 | 5.00 | ON |

*Table 1.4.1 Effect of Potentiometer on Output Voltage and LED State*

The relationship between the ADC readings and output voltage was determined by using the following formula:

$$V = \frac{ADC\ Value}{1023} \times 5$$




*Graph 1.4.1 Relationship Between ADC Readings and Output Voltage*

## 1.5 DATA ANALYSIS

The data obtained from this project is based on the state of the LED state in response to its user inputs from the potentiometer. As the potentiometer knob was turned clockwise, the ADC value increased and the voltage increased proportionally.

As the condition to turn ON the LED is that the ADC value must be more than half of 1023, which is 511.5, then the minimum voltage to turn on the LED is 2.5 V. Otherwise, the LED will turn OFF.

In conclusion, the results demonstrate that the interfacing between the microcontroller, potentiometer, and the LED is successful. The system can perform both ON and OFF operations, reflecting the successfulness of transmitting potentiometer readings from Arduino to Python, and Python send a command back to Arduino to turn ON or OFF the LED as required.

## 1.6 RESULT

The results of this project are shown in the figures, where the serial monitor in PyCharm successfully display the value of potentiometer, output voltage and the resistance value. The LED is operated correctly based on the limited condition. When the value of the potentiometer is more than 511.5, the LED will turn ON, while the LED will turn OFF when the value of the potentiometer is less than 511.5. The figures below clearly show the system is functioning as required.

*Fig 1.6.1 The Serial Monitor Display the Results*

*Fig 1.6.2 The Serial Monitor Display the Results*

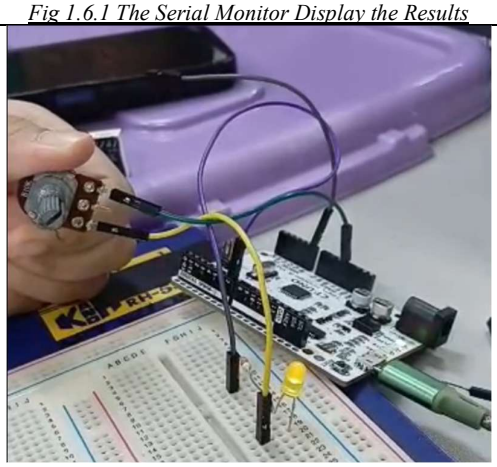


*Fig 1.6.4 The LED Turn ON*

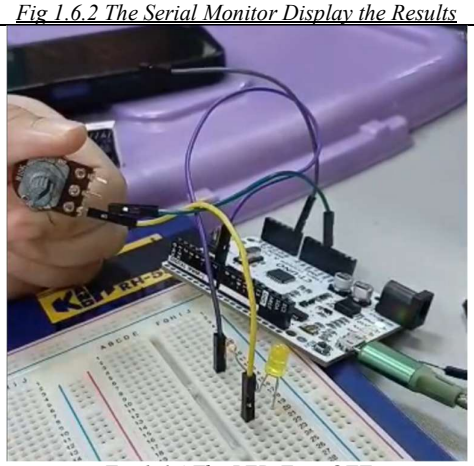


*Fig 1.6.4 The LED Turn OFF*

## 1.7 DISCUSSION

The results obtained from this experiment show a clear relationship between the potentiometer input and the LED output, demonstrating the effectiveness of the system design. As the potentiometer was rotated clockwise, the ADC value increased proportionally, indicating a higher input voltage being read by the Arduino. This analog signal was successfully converted into digital values and transmitted to Python through serial communication. Based on the predefined condition, the LED turned ON when the ADC value exceeded 511.5, which corresponds to approximately 2.5 V, and remained OFF when the value was below this threshold. This confirms that the system accurately interprets input data and applies conditional logic to control the output.

Furthermore, the results displayed in the PyCharm serial monitor verify that the potentiometer values, output voltage, and resistance were correctly measured and transmitted. The bidirectional communication between Arduino and Python functioned as expected, where Python processed the incoming data and sent control commands back to the Arduino to operate the LED. The consistent ON and OFF behavior of the LED based on the input condition indicates that the integration between hardware and software was successful. Overall, this experiment demonstrates reliable data acquisition, processing, and control, highlighting the effectiveness of serial communication in embedded systems.

## 1.8 CONCLUSION

In summary, this project has been completed effectively by achieving its objective to transmit the potentiometer readings from the Arduino to Python via USB serial communication and operating

bidirectional communication. The results show that as the potentiometer value varied from 0 to 1023, the corresponding voltage readings were generated and can be monitored.

Additionally, the LED state verified the presence of a switching threshold, where the LED remained OFF at lower voltages and turned ON at approximately 2.5 V. This illustrates that an analog signal can be used to control a digital output based on a predefined condition.

Overall, the results of this project have achieved the expectations where the analog sensor data can be read by Arduino and transmitted the data to Python for further processing.

## TASK 2

### 2.1 MATERIALS AND EQUIPMENTS

- Arduino board + USB cable
- Potentiometer
- Jumper wires
- Servo motor
- Breadboard

### 2.2 EXPERIMENTAL SETUP

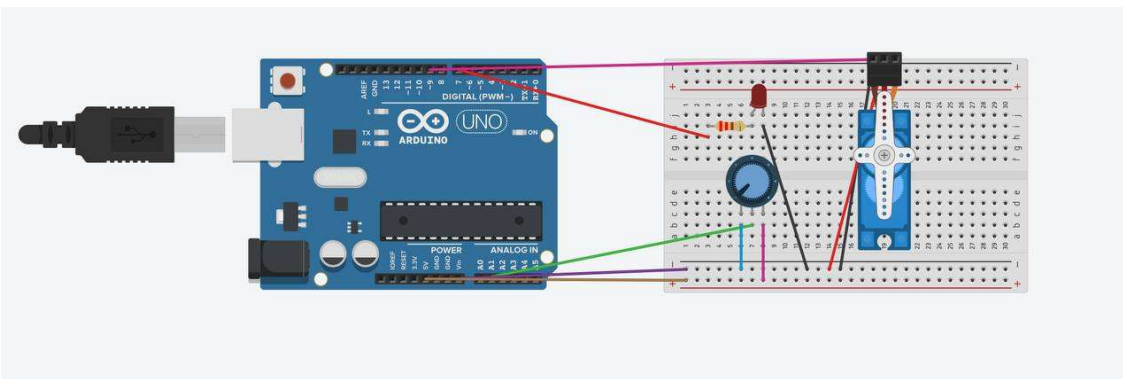



*Figure 2 illustrates the project setup constructed using TinkerCad*

1. The servo motor's signal pin is connected to Arduino digital pin 9, VCC to 5V, and GND to GND.
2. The potentiometer's pin is connected to analog pin A0.
3. The LED is connected in series with a 220 Ohm resistor to pin 7 of Arduino.
4. The Arduino code is being uploaded to handle the hardware logic.
5. By running the Phyton script, the data will be visualized. As the potentiometer being turn, the servo motor will be rotated. The LED will turn on if exceeds 90 degrees. The graph will be shown on the screen.

## 2.3 METHODOLOGY

### a. Circuit Assembly

- All components were assembled based on the given circuit setup and wiring instruction (Fig.1).
- The wiring was ensured for proper continuity and to prevent short circuits.
- The Arduino Uno was connected to the computer by using a USB cable for power and data transfer.

### b. Programming and Uploading

- The Arduino IDE software was opened on the computer.
  - The correct board type (Arduino Uno) and communication port (COM port 7) were selected in the Tools menu.
  - The coding was uploaded and compiled with no syntax errors.

- The latest version of Python and PyCharm were downloaded on the computer.
  - During installation, ensure to check the box that says, "Add Python to PATH".
  - Install the required libraries by typing this single command and pressing Enter:
    `pip install pyserial matplotlib`
  - The Python script was uploaded and executed. To stop the program safely, click the red "Stop" square in PyCharm or press CTRL+C in the terminal window.
  - Ensure that Arduino IDE is not opened when running Python code in PyCharm.

### c. Programming Logic

- Arduino IDE
  - The potentiometer, servo motor, and LED pins were initialised at the beginning of the program.
  - The Arduino continuously reads the analog value from the potentiometer through analog pin A0.
  - The analog value (0–1023) was mapped into a servo motor angle range from 0° to 180°.
  - The servo motor position was updated continuously according to the mapped angle value.
  - The LED acted as an indicator, where the LED turned ON when the servo angle exceeded 90° and turned OFF when the angle was below 90°.
  - The Arduino sent the servo angle data to Python through serial communication for real-time monitoring and plotting.
  - The Arduino also listened for a stop command ('x') from Python. When the command was received, the servo motor was detached, the LED was turned OFF, and the system stopped safely.

1. Python (PyCharm)
   - Python established serial communication with Arduino through the specified COM port.
   - The servo angle data sent from Arduino was continuously read through the serial port.
   - The received angle values were displayed in a live table format showing the sample number and angle value.
   - The angle data was plotted in real time using matplotlib to create a real-time servo angle graph.
   - A sliding window method was used to display only the latest 200 samples so that the graph remained clear and readable.
   - The graph was continuously updated without clearing the entire plot to ensure smooth real-time visualization.
   - When the user pressed CTRL+C, Python sent a stop command ('x') to Arduino to safely stop the servo motor and close the serial connection.

## d. Coding

**Arduino IDE:**

sketch_mar25a.ino

```
#include <Servo.h>

Servo myservo;
const int potPin = 9;  //Potentiometer pin
const int ledPin = 7;
bool stopControl = false;
int potValue, angle;

void setup() {
  Serial.begin(9600);
  myservo.attach(9);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  if (!stopControl) {
    potValue = analogRead(potPin);

    // Map the 0-1023 potentiometer value to a 0-180 degree angle
    angle = map(potValue, 0, 1023, 0, 180);
    myservo.write(angle);

    // LED control: Turn ON when the angle exceeds 90 degrees
    if (angle > 90) {
      digitalWrite(ledPin, HIGH);
    } else {
      digitalWrite(ledPin, LOW);
    }

    // Send the calculated angle to Python
    Serial.println(angle);
    delay(50);
  }
```

```
    }

    // Send the calculated angle to Python
    Serial.println(angle);
    delay(50);
  }

  // Listen for the stop command ('x') from Python
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'x') {
      stopControl = true;
      myservo.detach();            // Safely detach the servo
      digitalWrite(ledPin, LOW);   // Turn off the LED
      Serial.println("STOP");
    }
  }
}
```

**Python (Pycharm):**

```
import serial
import matplotlib.pyplot as plt
import time

# Initialize Serial
ser = serial.Serial( port: 'COM7',  baudrate: 9600)
time.sleep(2)

# Initialize Plot
plt.ion()
fig, ax = plt.subplots()
x_vals, y_vals = [], []
line, = ax.plot( *args: [], [], 'b-')  # Create a line object

# Set axis limits once
ax.set_ylim( bottom: 0,  top: 180)
ax.set_xlim( left: 0,  right: 200)
ax.set_title("Real-Time Servo Angle Plot")
ax.set_xlabel("Samples")
ax.set_ylabel("Angle (°)")

print(f"{'Sample':<10} | {'Angle':<10}")
print("-" * 25)

try:
    while True:
        if ser.in_waiting > 0:
            raw = ser.readline().decode().strip()

            if raw.isdigit():
                angle = int(raw)
```

```python
                angle = int(raw)

                # THIS IS THE LIVE TABLE DATA DISPLAYING BELOW
                print(f"{len(x_vals):<10} | {angle}°")

                x_vals.append(len(x_vals))
                y_vals.append(angle)

                # Sliding window (last 200 points)
                if len(x_vals) > 200:
                    y_vals = y_vals[-200:]
                    x_vals = list(range(200))  # Keep x-axis stable

                # Update the line data instead of clearing the whole plot
                line.set_data(range(len(y_vals)), y_vals)

                # Refresh the canvas
                fig.canvas.draw()
                fig.canvas.flush_events()

        plt.pause(0.01)  # Small pause to allow the GUI to process

except KeyboardInterrupt:
    ser.write(b'x')
finally:
    ser.close()
    plt.ioff()
    plt.show()
```

## 2.4 DATA COLLECTION

| Potentiometer rotation | Servo angle (◦) |
|---|---|
| 83 | 121 |
| 86 | 104 |
| 98 | 72 |
| 100 | 66 |
| 102 | 64 |

## 2.5 DATA ANALYSIS

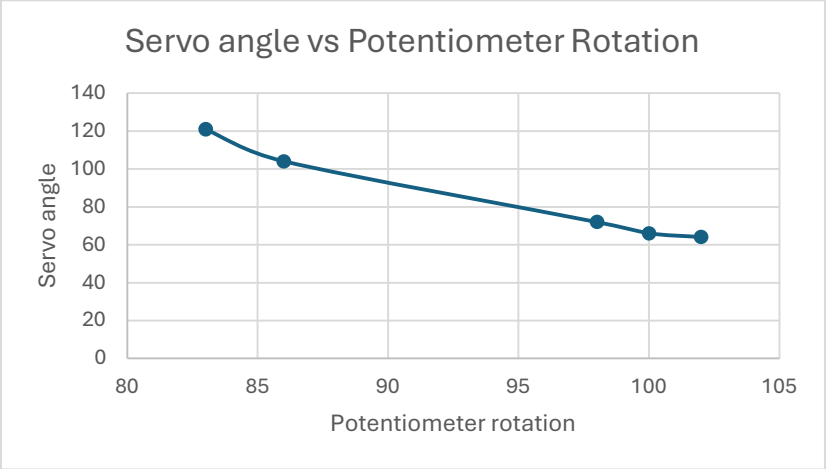



*Figure 3 shows the relationship between servo angle and potentiometer rotation*

Based on the graph, we can see that the Servo angle is inversely proportional to the Potentiometer Rotation. The potentiometer increases from 83 to 102, and the servo motor angle decreases from 121 to 64. This shows that the Arduino/Python has used a mapping function that is going to invert the input to output direction. The control logic based on the graph is that as the physical rotation of the potentiometer (input) increases, the lower the resulting angular position of the servo motor (output).

Other than that, the data also shows us the linearity as they follow nearly a straight line. This proves that the system maintains stable linearity. However, there is a slight curve at the end of the graph. This might indicate that the servo motor has a physical limitation in the servo motor or a potentiometer where small input changes no longer produce significant output shifts.

## 2.6 RESULT

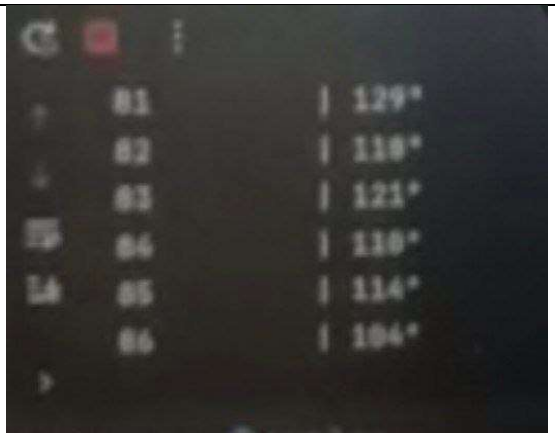



*Figure 4 shows **Input 83:** Servo angle at 121° and **Input 86:** Servo angle at 104°.*

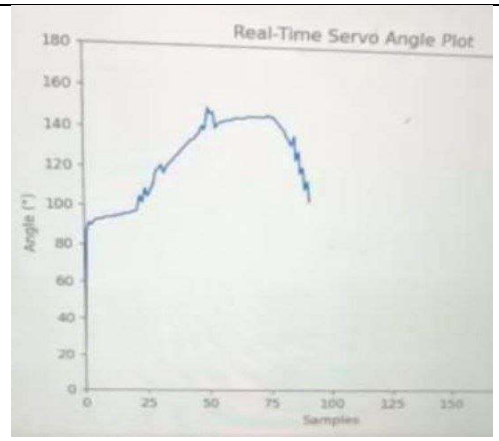



*Figure 5 shows the graph for the data in Figure 4*

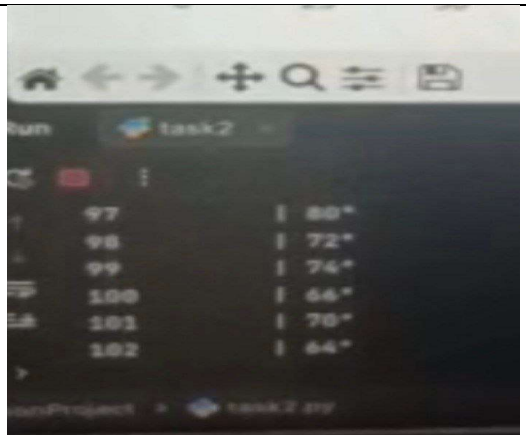



*Figure 6 shows Input 98: servo angle 72°, Input 100 servo angle 66°, Input 102: servo angle 64°*

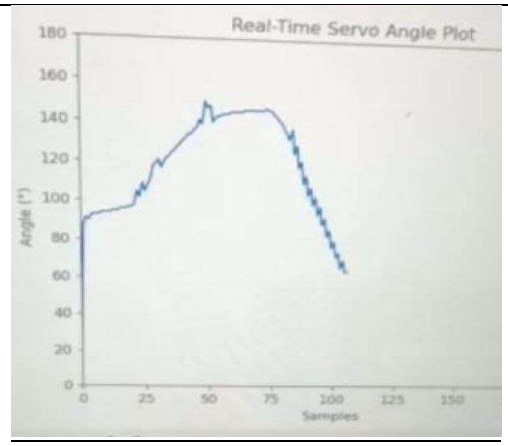



*Figure 7 shows the graph for data in Figure 6*

## 2.7 DISCUSSION

The experiment shows a linear inverse relationship between the potentiometer rotation (input) and the servo motor angle (output). It can be achieved by implementing the map () function inside the Arduino IDE code. At row 20, map (pot Value, 0, 1023, 0, 180) shows that the software's function is going to translate the raw 10-bit analogue signal from 0 to 1023 into a specific angle range 0° to 180°. This task also implements the loop concept. According to our code, Arduino IDE, the system serves as a threshold indicator since we are using if (angle > 90). The LED will trigger when the potentiometer is rotated past the midpoint of the mapping. This mechanism is what we called as control system as the servo and the LED respond simultaneously to a single sensor input.

Based on our data, the system has high sensitivity. The servo angle changed by 17° with a mere 3 unit increase in potentiometer rotation (from 83 to 86). This sows that the control system's resolution is fine enough to pick up on manual adjustments, enabling accurate actuator positioning. Other than that, the zig-zag edges observed in the Real – Time Servo Angle Plot is indication of Analog jitter.

The electrical line can occur when a line that should be flat seems fuzzy or thick. It shows us that the signal isn't quite even when you are not touching the knob. The phyton script connects to the Arduino over a 9600-baud communication channel using the pyserial module. The attempted update rate of 100 Hz is indicated by the line plt.pause (0.01). Based on figure 5 and figure 6, each tiny millivolt variation from the potentiometer is being recorded and shown quickly at 100 updates per second.

## 2.8 CONCLUSION

In conclusion, by connecting the Arduino to a Python-based graphical user interface, this project taught us how to integrate hardware and software to create real-time mechatronic control and data visualisation. Certain objectives are being achieved by completing the task. One of them is a real-time control loop. Using the map () function, the Arduino continually sampled analogue input from a potentiometer and converted those values (0-1023) to exact servo motor angles (0-180).

Next is Bidirectional communication, where both platforms were maintained by successful serial communication. Python was still able to issue a halt instruction, but the Arduino sent calculated angle data to Python for monitoring.

Last but not least, Automated Conditional Logic is also being taught for this task. The system demonstrates basic automated decision-making by controlling an LED based on the servo's position.

## 3.0 RECOMMENDATION

For **Task 1**, one recommendation is to implement signal filtering to improve the stability of the potentiometer readings and overall system performance. This is because the potentiometer readings may fluctuate due to electrical noise or small mechanical movements when adjusting the knob during the experiment. This may cause an unstable ADC value, which may result in the LED switching ON and OFF rapidly when the value is near the threshold. For example, in inductive debris sensing, a digital potentiometer is used to tune the filter characteristics in real time to match changing flow rates, greatly improving SNR and reducing signal standard deviation compared to a fixed low-pass filter. Thus, for a potentiometer, we can consider using a tunable cutoff like adjustable $\alpha$ in the digital low-pass, so that the input signal can be stabilised and produce more consistent output behaviour.

In addition, the reliability of serial communication between Arduino and Python can also be enhanced by improving system stability. The stable input readings will ensure that the data sent to Python is smoother and more accurate, reducing sudden jumps in displayed values. Therefore, this will improve the overall performance of the system, which produces a more reliable hardware-software integration. In conclusion, signal filtering and system stability improvement are important to ensure accurate sensor readings and stable output control.

For **Task 2**, the implementation of signal filtering to improve servo motor smoothness and real-time data visualisation. A smooth servo motion and accurate real-time plotting depend on clean and stable input signals with minimal noise and delay. During the experiment, the noise from the potentiometer readings can cause small fluctuations in the servo angle, resulting in jittery servo movement and unstable graph plotting. Therefore, by implementing filtering techniques like moving average filters or low-pass filters, the noise in the signal can be reduced, which produces smoother servo motion and cleaner real-time plots. This will improve the overall performance of the control system and make the data visualisation more accurate and easier to analyse.

Moreover, filtering must be carefully designed to maintain system stability and avoid excessive delay in the control response. This is because a poorly designed filter may introduce phase lag, which can reduce system responsiveness and affect servo performance. Consequently, tuned filters such as low-pass filters or observer-based filtering methods can be used to reduce noise while maintaining system stability and tracking accuracy. Then, the servo motor system can achieve smoother motion, better tracking performance, and more stable real-time graphical visualisation.

## 4.0 REFERENCES


Arduino Reference (2026). *Mathematical Functions: map()* https://www.arduino.cc/reference/en/language/functions/math/map/

*PySerial Documentation (2026).*

https://pyserial.readthedocs.io/

*Devan, P., Hussin, F., Ibrahim, R., & Bingi, K. (2021). Design and Implementation of Signal Filtering Techniques on Real-time Pressure Process Plant. 2021 11th IEEE International Conference on Control System, Computing and Engineering (ICCSCE), 86-91.* https://doi.org/10.1109/iccsce52189.2021.9530979.

*Wang, G., Wu, X., Qian, Z., Liu, D., Li, P., & Kuznetsova, I. (2024). A novel demodulator design for signal processing of inductive oil debris sensors. Measurement Science and Technology, 36.* https://doi.org/10.1088/1361-6501/ad894a.

*Dursun, C., & Ozkilic, S. (2025). A Novel Filtering Observer: A Cost-Effective Estimation Solution for Industrial PMSM Drives Using in-Motion Control Systems. Energies.* https://doi.org/10.3390/en18040883.


## 5.0 APPENDICES

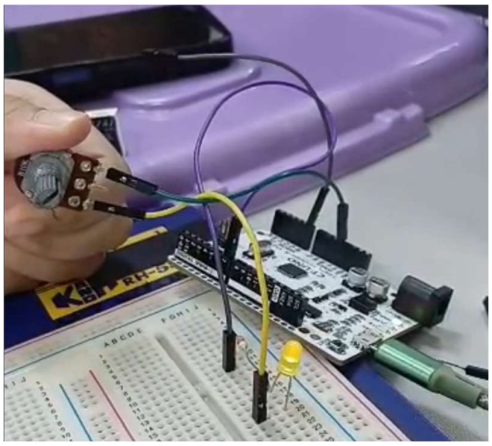


*Figure 5.1 shows the circuit assembled on a breadboard following the designed connection layout of* ***Task 1****.*

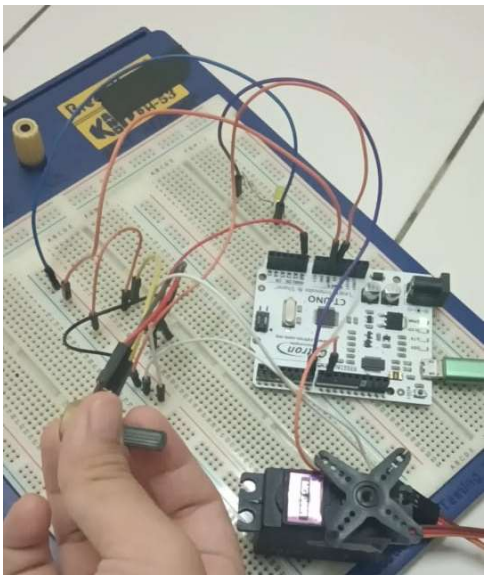


*Figure 12.2 shows the circuit assembled on a breadboard following the designed connection layout of **Task 2**.*

## 6.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help, encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

## 7.0 STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behaviour. Every information and support source used in this work has been appropriately referenced and acknowledged.

***Certificate of Originality and Authenticity***

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributor to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final report has been verified by us.

Signature: nurulanis
Name: NURUL ANIS BINTI MOHD KHAIRI
Matric Number: 2419788

Read [/]
Understand[/]
Agree [/]

Signature: athirahmeor
Name: WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN
Matric Number: 2320102

Read [/]
Understand[/]
Agree [/]

Signature: nurkhalisah
Name: SITI NURKHALISAH BINTI NAHZAMRI
Matric Number: 2328482

Read [/]
Understand[/]
Agree [/]